

Proyecto Realizado por:
David Santiago Tellez Melo 20242020107
Johan Nicolas Lopez Hernandez 20242020126

DEVELOPMENT AND DESIGN OF A MICROBLOGGING PLATFORM:VIBE

OBJECTIVES

provide an easy-to-understand interface

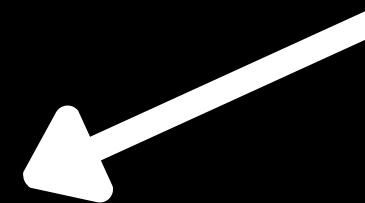
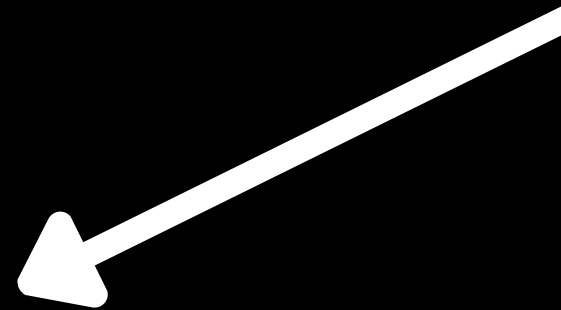
Possibility to publish texts of up to 280 characters, links and images.

Buttons to which users can interact with each other.

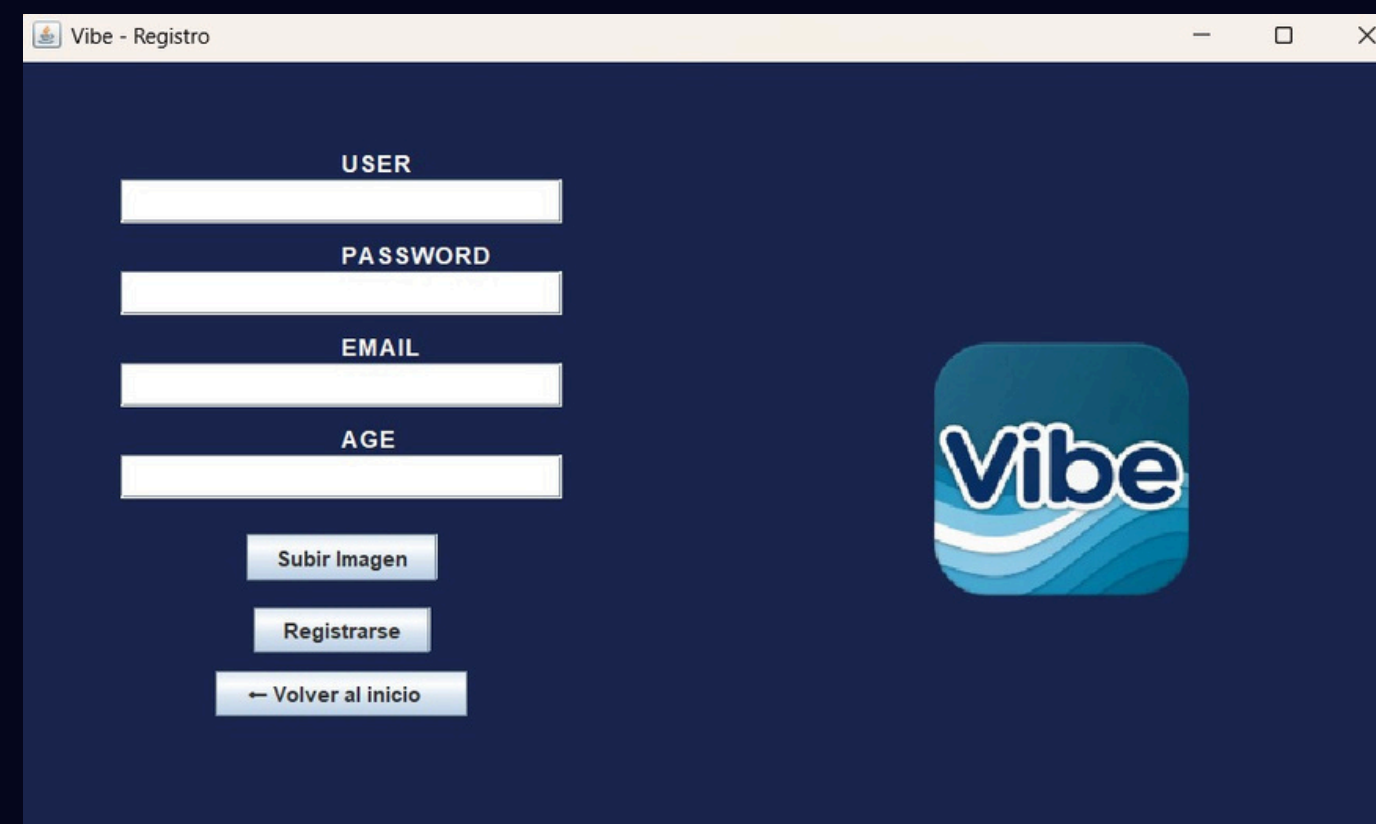
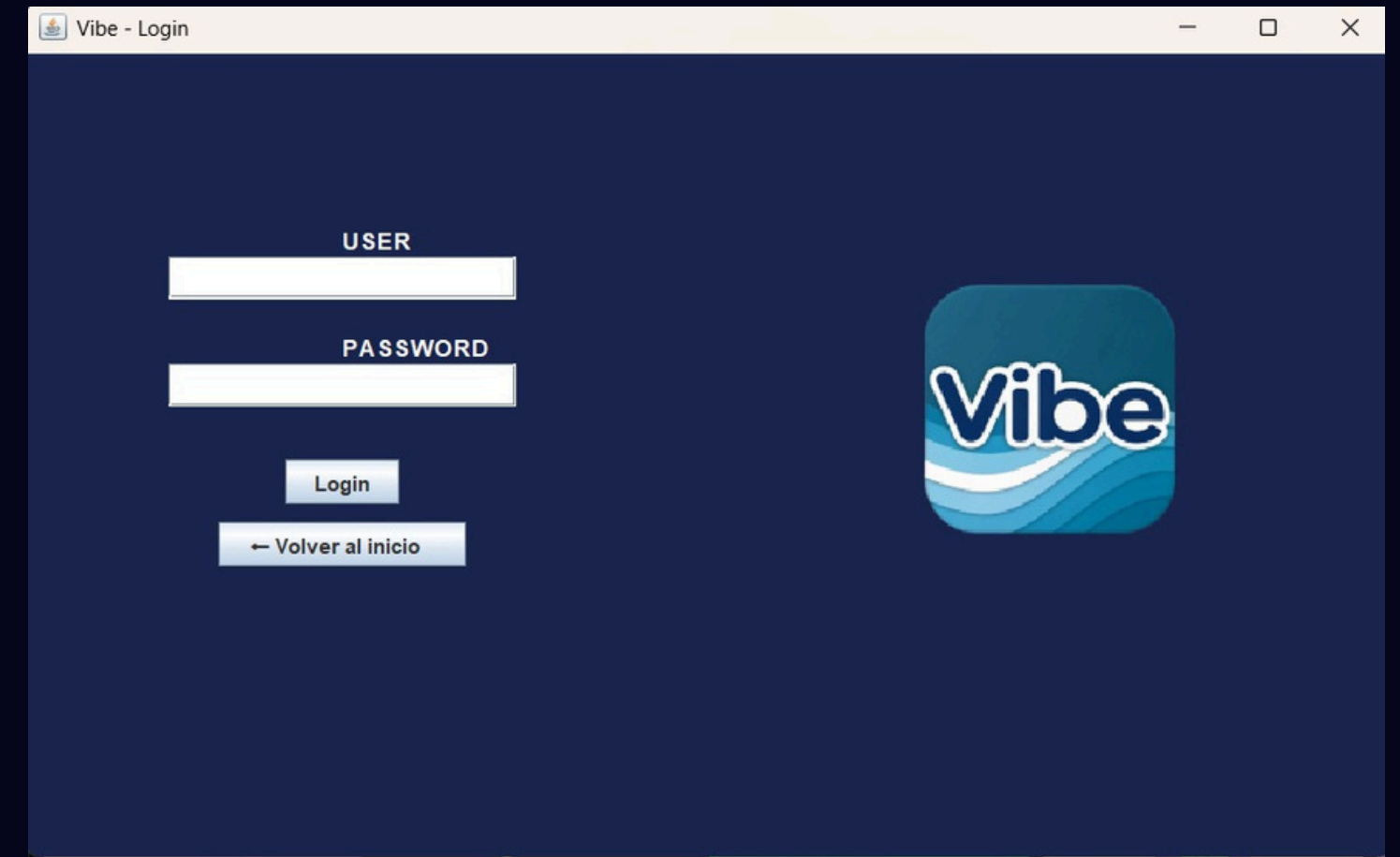
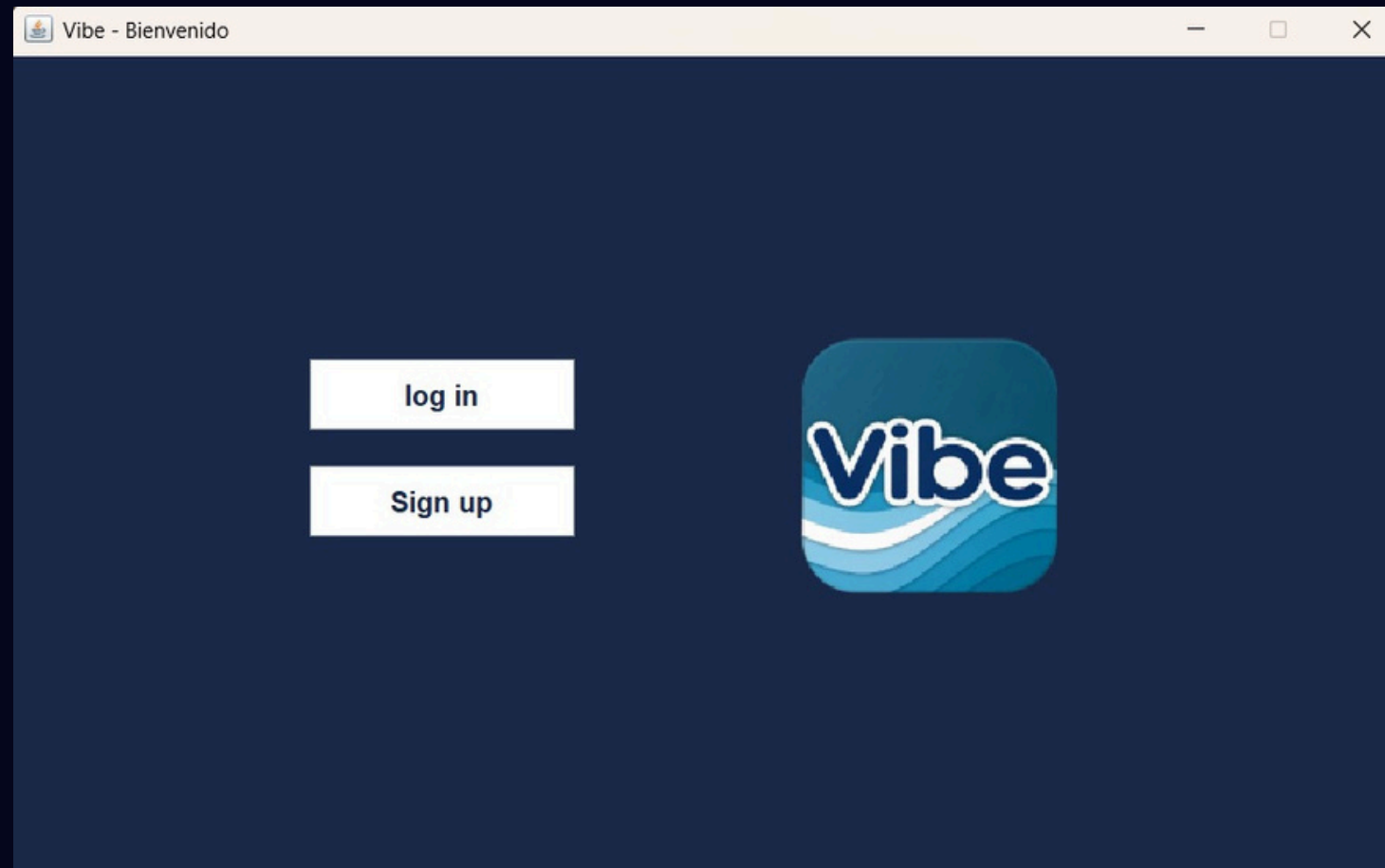
It should be possible to enter basic user information:

name, age, email, profile picture, among others.

Have a main feed where you can see the content of other users.



PROJECT



Methodological approach

Object-Oriented-Programming



Modularity

Model

Scalability

View

Maintenance

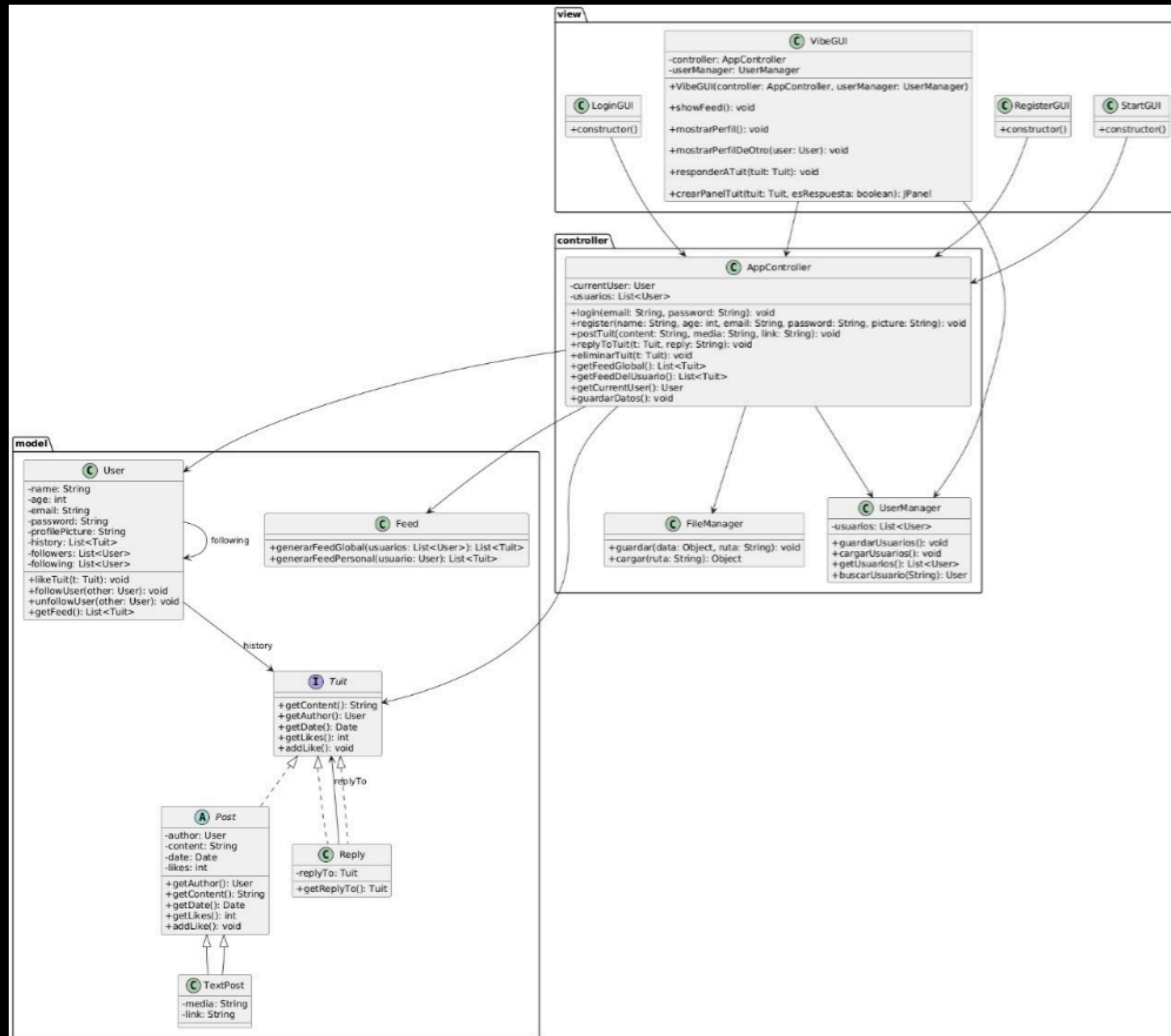
Controller

Desing tools

Class FileManager		Class VibeGUI		Interface: Tuit	
Responsibilities	Collaborations	Responsibilities	Collaborations	Responsibilities	Collaborations
- Read and write persistent data to disk (JSON, images). - Support loading of users, tuits, and media.	- UserManager - AppController - User - Post - Reply	- Display the main interface (feed, buttons, input forms). - Allow user to create tuits, replies, and likes. - Display profile views. - Interact with AppController for logic execution.	- AppController - UserManager - Tuit - User	- Define common attributes and behaviors of any tuit. - Abstract access to author, content, date, and likes.	- Post - Reply
Class Reply		Class UserManager		Class Post	
Responsibilities	Collaborations	Responsibilities	Collaborations	Responsibilities	Collaborations
- Represent a reply to another tuit. - Store content, timestamp, likes. - Maintain reference to original tuit. - Track its author.	- Tuit (as parent) - User - AppController	- Handle user registration and authentication. - Maintain list of all users. - Validate credentials. - Save and load user data from persistent storage.	- User - FileManager - AppController	- Represent a standalone tuit (text post). - Store content, timestamp, likes, optional media and link. - Track its author.	- User - AppController
Class User		Class AppController		Class LoginGUI	
Responsibilities	Collaborations	Responsibilities	Collaborations	Responsibilities	Collaborator
- Store user attributes: name, email, age, password, profile picture. - Manage lists of followers and followed users. - Publish posts and replies. - Like and delete own tuits. - Follow and unfollow other users.	- Post - Reply - UserManager - AppController	- Mediate between model and GUI. - Manage tuit publication, reply, like, and deletion. - Handle current session (logged-in user). - Access data for feed and user-specific actions. - Save and load persistent data.	- User - Post - Reply - FileManager - UserManager - VibeGUI	- Allow users to log in with email and password. - Validate credentials and redirect to main app.	- UserManager - AppController
Class Feed		Class RegisterGUI		Class StartGUI	
Responsibilities	Collaborator	Responsibilities	Collaborator	Responsibilities	Collaborator
- Build and return the general feed (all posts). - Sort post chronologically	- User - Post - Reply - FileManager	- Let new usuer sing up with basic data and profile picture. - Create and sabe the new user.	- AppController - UserManager - FileManager	Display the welcome screen with "Login" and "Register" options	- LoginGUI - RegisterGUI

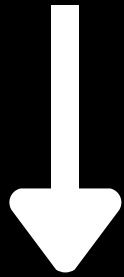


Desing tools



General architecture

Model



Interface: tuit
Feed
User
Reply
Post

View



LoginGUI
RegisterGUI
StarGUI
VibeGui

Controller



AppController
FileManager
UserManager

Model classes

Class User	
Responsibilities	Collaborations
• Store user attributes: name, email, age, password, profile picture. • Manage lists of followers and followed users. • Publish posts and replies. • Like and delete own tuits. • Follow and unfollow other users.	• Post • Reply • UserManager • ApplicationController

Class Post	
Responsibilities	Collaborations
• Represent a standalone tuit (text post). • Store content, timestamp, likes, optional media and link. • Track its author.	• User • ApplicationController

Class Feed	
Responsibilities	Collaborator
• Build and return the general feed (all posts). • Sort post chronologically	• User • Post • Reply • FileManager

Class Reply	
Responsibilities	Collaborations
• Represent a reply to another tuit. • Store content, timestamp, likes. • Maintain reference to original tuit. • Track its author.	• Tuit (as parent) • User • ApplicationController

Interface: Tuit	
Responsibilities	Collaborations
• Define common attributes and behaviors of any tuit. • Abstract access to author, content, date, and likes.	• Post • Reply

View classes

Class StartGUI	
Responsibilities	Collaborator
Display the welcome screen with “Login” and “Register” options.	- LoginGUI - RegisterGUI

Class LoginGUI	
Responsibilities	Collaborator
- Allow users to log in with email and password. - Validate credentials and redirect to main app.	- UserManager - AppController

Class RegisterGUI	
Responsibilities	Collaborator
- Let new user sign up with basic data and profile picture. - Create and save the new user.	- AppController - UserManager - FileManager

Class VibeGUI	
Responsibilities	Collaborations
- Display the main interface (feed, buttons, input forms). - Allow user to create tweets, replies, and likes. - Display profile views. - Interact with AppController for logic execution.	- AppController - UserManager - Tweet - User

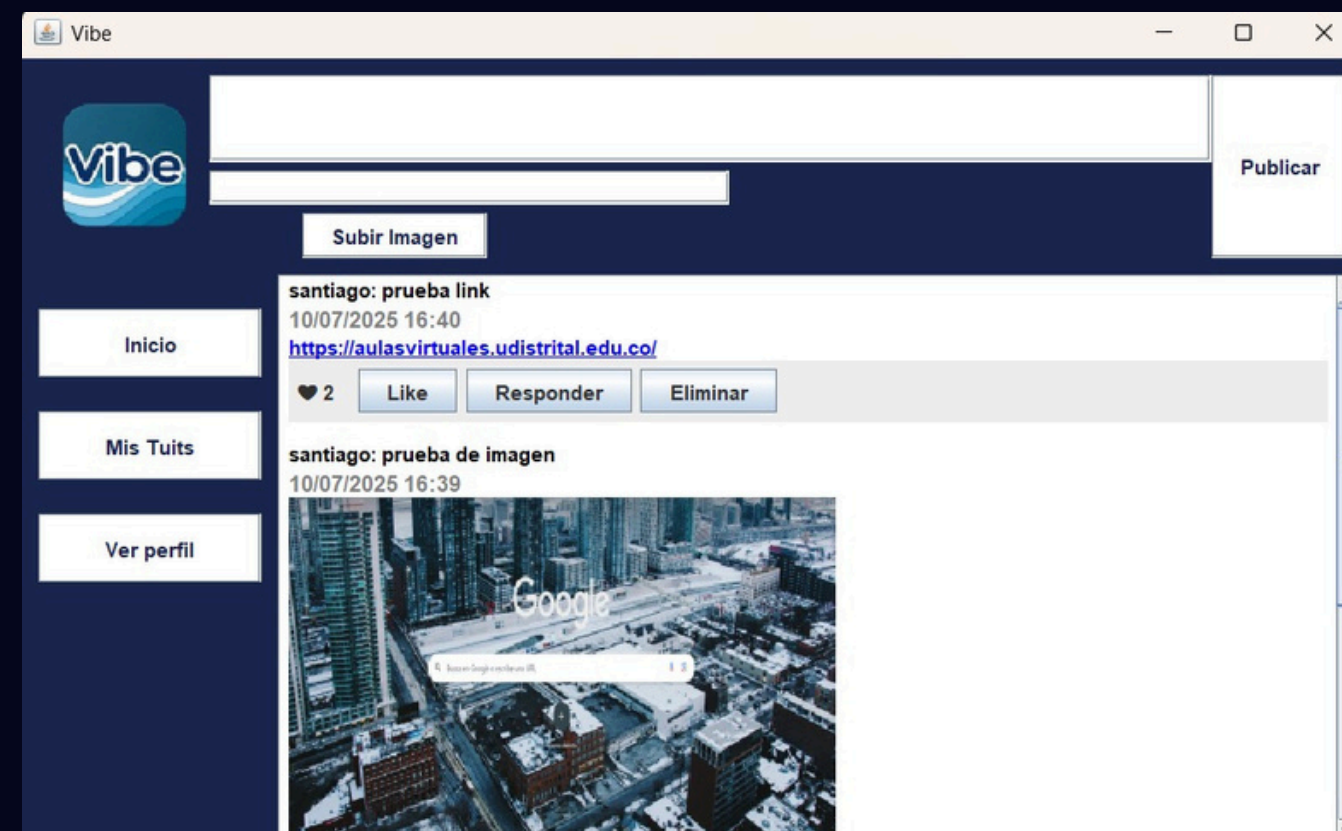
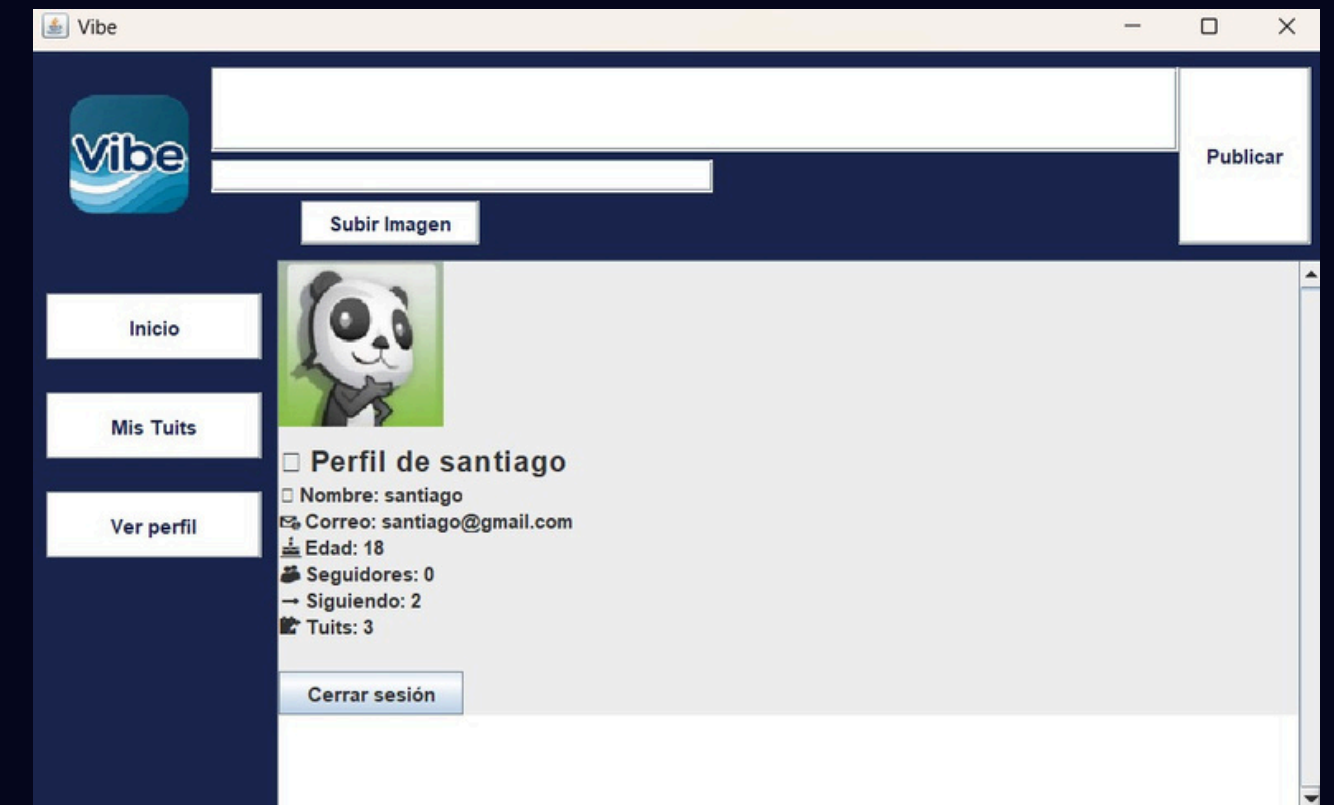
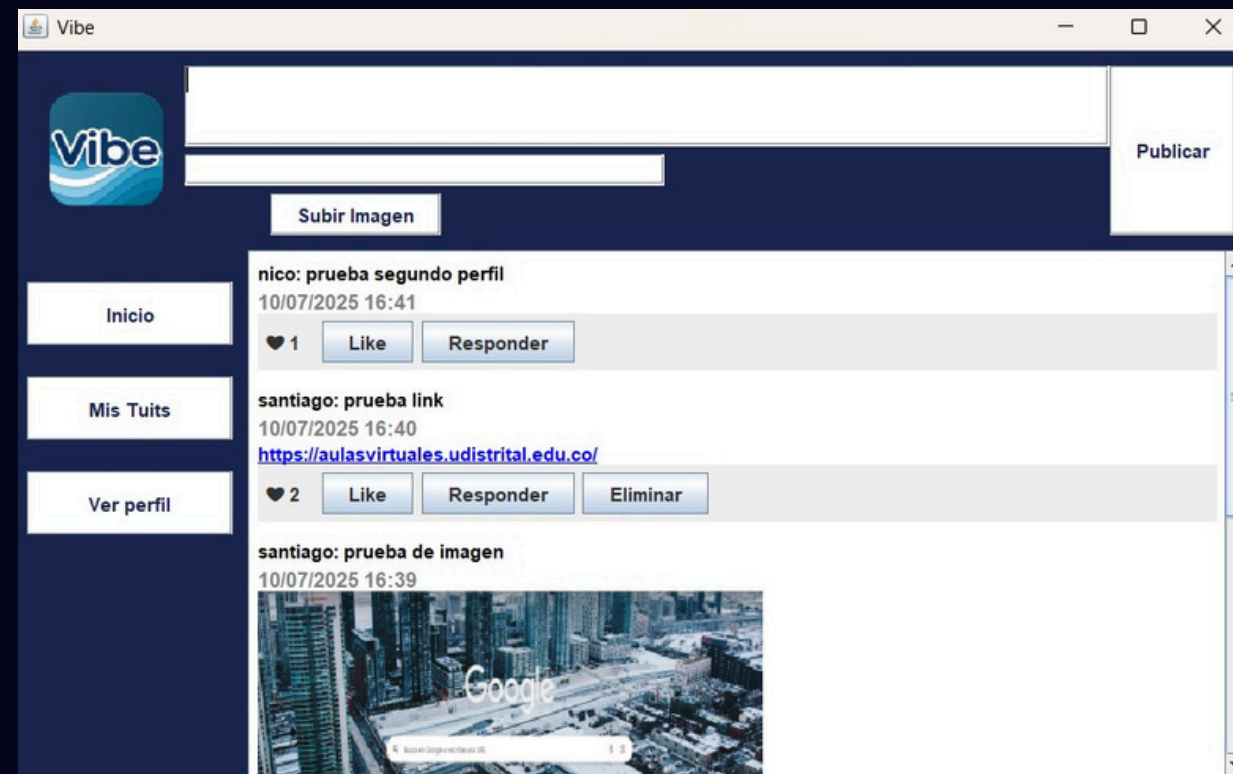
Controller classes

Class AppController	
Responsibilities	Collaborations
• Mediate between model and GUI. • Manage tweet publication, reply, like, and deletion. • Handle current session (logged-in user). • Access data for feed and user-specific actions. • Save and load persistent data.	• User • Post • Reply • FileManager • UserManager • VibeGUI

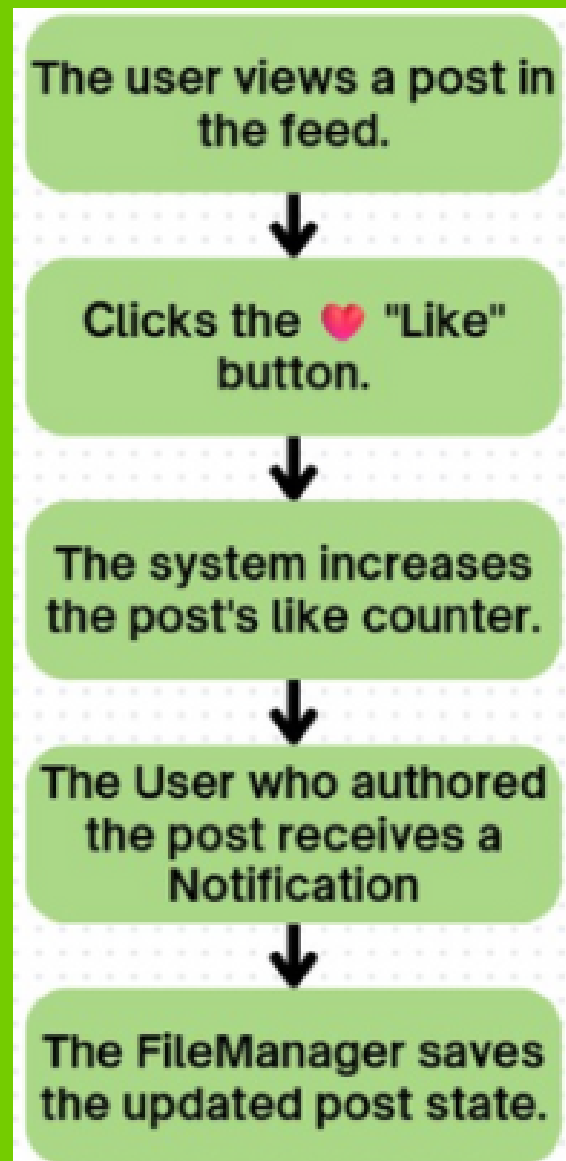
Class UserManager	
Responsibilities	Collaborations
• Handle user registration and authentication. • Maintain list of all users. • Validate credentials. • Save and load user data from persistent storage.	• User • FileManager • AppController

Class FileManager	
Responsibilities	Collaborations
• Read and write persistent data to disk (JSON, images). • Support loading of users, tweets, and media.	• UserManager • AppController • User • Post • Reply

System functionalities



Use cases



Conclusions

The Vibe project demonstrated the feasibility of designing a microblogging platform using object-oriented programming and modular architecture.

The application integrates key functionalities such as user registration, secure authentication, creation of posts with diverse content, feedback system and visualization of a shared feed. Through the use of the MVC architecture pattern, together with abstraction and encapsulation principles, a clear separation of responsibilities was achieved, favoring the scalability and maintenance of the system

Although it runs locally without cloud support or real-time communication, it constitutes a robust proof of concept for academic purposes. Future improvements such as incorporation of comments, mobile version, persistence through database and optimization of the interface are contemplated, with the aim of bringing the application closer to current standards of interactive social platforms.